RTDL: A Restricted-Python DSL for Repurposed RT

Anonymous Author(s)

Abstract

Ray-tracing (RT) hardware increasingly executes non-rendering workloads in which a geometric hit denotes an application candidate, record, neighbor, or aggregate contribution. Such meanings impose obligations on coverage, identity, repeated delivery, termination, and reduction. These obligations cross source callbacks, data and geometry interpretation, physical traversal, and result return; local stage legality and machine types do not capture the whole relation.

We present RTDL, a restricted-Python domain-specific language and compiler for bounded repurposed RT computations. Authors declare immutable records and typed role functions. The compiler constructs canonical Callback IR and checks role effects, resources, semantic and physical contracts, and supported result routes. Supported authored templates generate a deterministic ABI and Numba leaves for trusted topology-specific CUDA/OptiX wrappers; closed routes can instead select dedicated native implementations. Prepared execution binds the admitted program, target, and runtime identities. For fixed families, a result obligation changes concrete compiler decisions: complete-relation routes reject unsupported early termination, logical all-hit acceptance generates a payload update followed by physical intersection rejection, and exact callback IR selects guarded standard-route specialization.

The implementation exposes two stable constructors and a bounded corpus over triangle, custom-AABB, curve, and sphere leaves. A historical audit covers nine project-authored V4 mappings. An initial public-PyOptiX comparison covered selected stages from three mappings (four operation units); all twelve setup, first-execute-after-preparation, and prepared observations were slower (1.080–75.533×). A separately identified remediation then put all ten registered setup-after-shared-preprocessing/prepared rows over those units and one long graph scale within a 1.20×-median/1.35×-maximum engineering envelope; the 8.8–9.3 s graph row was 1.057×. Separately, two exact standard routes were 1.077–1.175× Direct when prepared. RTDL does not infer profitable mappings, accept arbitrary Python, synthesize arbitrary topologies, prove application semantics, or establish easier authoring or broad performance.

CCS Concepts: • **Software and its engineering → Compilers**.

Keywords: ray tracing, domain-specific languages, restricted Python, GPU compilation

ACM Reference Format:
Anonymous Author(s). 2027. RTDL: A Restricted-Python DSL for Repurposed RT. In *Proceedings of IEEE/ACM International Symposium on Code Generation and Optimization (CGO '27).* ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Dedicated RT traversal is now used for particle tracking, graph processing, spatial indexing and joins, neighbor search, clustering, database processing, and N-body simulation [6, 7, 19, 21, 22, 30, 35, 37, 39]. These systems show that a bounding-volume hierarchy can accelerate much more than image synthesis. They also expose a recurring programming problem. A hardware hit may be only a candidate, one logical row, a neighbor, or one contribution to a checked aggregate. The corresponding requirements for filtering, logical identity, repeated delivery, early termination, and reduction differ by result.

Prior systems already solve concrete instances; Table 1 separates three such mechanisms from the limited compiler responsibility studied here. LibRTS additionally exposes point/range handlers for count and collection [7]. These works motivate, rather than constitute, our contribution.

The language question is how a programmable interface can represent selected obligations so that they constrain admissible callbacks and generated execution.

Modern RT systems already provide substantial machinery. OptiX exposes the callback protocol; PyOptiX and SlangPy bring RT construction to Python; OWL and Luisa automate pipeline construction; DXR payload qualifiers and Slang capabilities enforce stage and target constraints [15, 20, 25–27, 31, 33, 38]. Rendering languages also connect domain results to renderer-owned execution; for example, Open Shading Language separates closure construction from integrator evaluation [1]. Our claim is not that rendering lacks protocols or that prior systems cannot express the same guarantee. We ask a narrower compiler question:

> For an author-selected RT mapping, how can a restricted high-level language express bounded non-rendering computation while making selected result obligations constrain callbacks, data interpretation, traversal actions, and result return?

RTDL answers for a bounded subset. Authors write immutable records, restricted role functions, and manifests

Table 1. Prior result protocols and RTDL's bounded connection; algorithms and RT mappings remain prior and app-owned.

Prior system	Result obligation and paper mechanism	Bounded RTDL connection
RayJoin [6]	Complete accepted-pair set; record each intersection, then physically ignore it to continue traversal	Fixed complete-relation admission rejects early termination and maps logical acceptance to physical ignore; predicates remain app-owned
RayDB [30]	One logical record despite repeated primitive delivery; deduplicate before aggregation	Historical bounded emission and grouped reduction; duplicate and SQL meanings remain app-owned
RTNN [39]	Range and nearest-neighbor queries require different bounded-result maintenance	Historical round/refit and bounded selection; metric, K-NN policy, and encoding remain app-owned

that name data, resources, and result contracts. The compiler builds typed Callback IR, checks cross-role and result-route constraints, generates leaf code and trusted topology wrappers, and binds the accepted executable to a prepared runtime. The application still owns the RT formulation, geometric encoding, domain predicates, and semantic oracle.

This paper makes four contributions:

1. We design a restricted-Python DSL and programming model for bounded repurposed RT. It exposes typed records and role-indexed effects while keeping raw PTX, SBT, pipeline, and traversal operations compiler-owned.
2. We develop typed Callback IR and whole-protocol admission that relate roles, result obligations, semantic and physical data contracts, resources, continuation, and executable identity. This is a fixed-family design, not automatic inference or a soundness proof.
3. We implement a deterministic ABI, generated Numba leaves, trusted topology-specific wrappers and exact-IR specialization, plus a prepared identity-checked runtime. The lowerers and specializers remain in the TCB.
4. We reconstruct nine project-authored V4 mappings and evaluate bounded mutation, composition, target checking, lifecycle, and performance. An adverse selected-stage 3/9 comparison motivates a separately identified deployment remediation; ten registered successor rows meet its engineering envelope, including one multi-second graph computation. Receipt, startup, coverage, generality, and authoring limits remain explicit.

2 RTDL by Example and Programming Model

Consider an application that has already selected nearest-hit queries over a triangle mesh. Mesh slots 0 and 1 belong to application objects 17 and 29; a query must return the object ID, a hit flag, and a semantic tag, while a miss returns (`0xffffffff`, 0, `MISS_TAG`). The lookup is deliberately simple: it exposes the programming boundary without claiming that RTDL discovers the RT mapping.

The complete authored program has a `Query` record containing origin, direction, and `tmax`; payload and output records each contain three `u32` fields. Its four roles are all explicit. `make_ray` reads a query and initializes the miss payload; `closest_hit` returns the first metadata value indexed by `hit.primitive_index`, with the hit flag and tag; `miss` preserves the payload; and `finalize` constructs the output record. Figure 1 traces that authored computation into its physical execution. Equal-distance, boundary, and degenerate cases are excluded from this illustration.

The manifest declares the records, constants, numeric and resource policy, built-in triangle geometry, and linkage. A physical plan names geometry, metadata, query, output, and status fields; binds the two metadata arguments to primitive-aligned columns $(17, 29)$ and $(31, 43)$; and supplies orientation and oracle identities. These are author responsibilities, not facts inferred from `u32`. The actual public host sequence calls `v4.verify_builtin_triangle_callback_source` and `v4.build_builtin_triangle_u32x3_physical_plan`; the verified program's `compile`, `materialize`, `prepare`, `execute`, and `close` methods connect target/toolchain, static mesh and metadata, and query batches. An existing test changes the closest-hit expression from mesh slot to metadata lookup: reusing the old source-bound plan is rejected, while a rebuilt plan changes the wrapper's actual metadata argument and is accepted. These CPU checks inspect generation and binding; they do not run this authored variant on a GPU.

The compiler owns parsing, verification, role/effect IR, ABI flattening, leaf code, trusted wrapper selection, PTX composition, program/SBT binding, status handling, and the prepared lifecycle. Users cannot inject raw traversal calls, arbitrary PTX, native callback pointers, or self-declared physical authority through the stable surface. Not every verified callback/topology pair has an executable lowerer; unsupported combinations fail closed.

Figure 2 expands the first row of Table 1 into one implemented fixed-family rule.

3 IR and Whole-Protocol Checking

3.1 Why callback typing is insufficient

A ray callback runs inside a protocol distributed across host and device code. Its legal effects depend on its role: an any-hit callback may ignore or terminate an intersection, whereas a miss callback cannot. Its payload fields must agree with every producer and consumer. Its program group must match the acceleration-structure leaf kind. The launch must occur only after module, pipeline, shader-binding-table, geometry, and output state have been

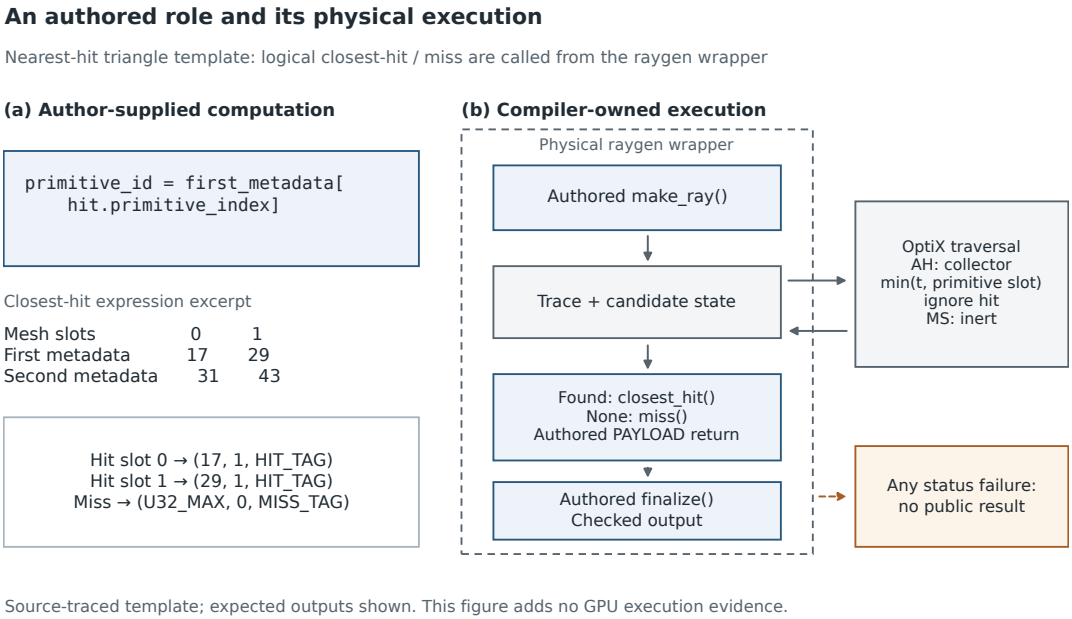

Figure 1. A complete-role authored triangle template, illustrated by its closest-hit expression and expected outputs. The compiler-owned wrapper selects the canonical candidate, calls exactly one logical closest-hit or miss leaf, then calls finalize. The other three roles are authored, not synthesized. The fixed template has three-u32 payload/output records and two metadata columns; source/binding CPU checks support this trace, not a new GPU execution.

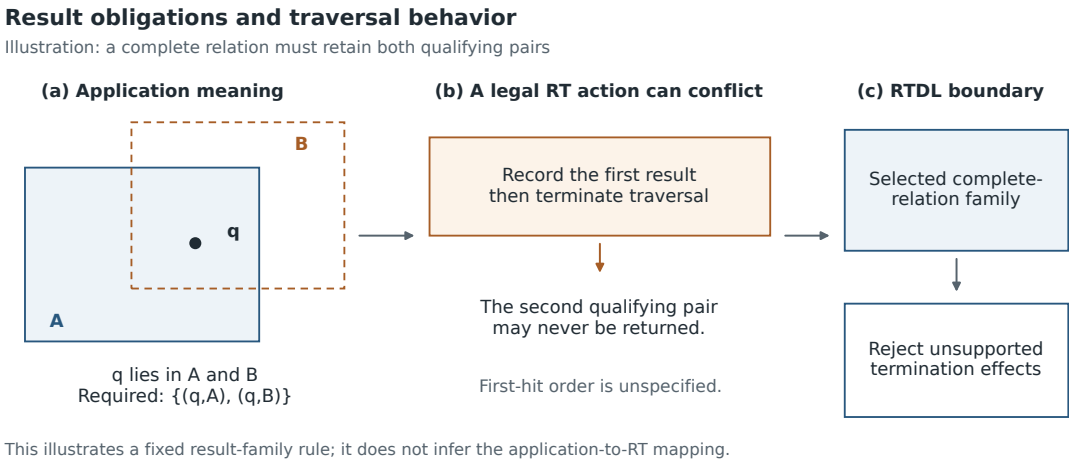

Figure 2. For a point contained in two objects, a complete relation contains both pairs. Early termination is a legal RT action but may omit one pair, so RTDL's bounded complete-relation family admits only acceptance with continued traversal. This is a fixed family rule, not a general completeness proof or an inferred application-to-RT mapping.

prepared. Finally, the code checked by the compiler must be the code loaded by the runtime.

These are related to typestate, interface automata, and session-style protocol reasoning [3, 13, 34], but the FFI and generated-code boundary adds an identity problem familiar from linking and foreign interfaces [5, 28]. Table 2 follows represented facts into concrete compiler consumers. Each row is an implemented source trace, not a theorem for arbitrary RT programs.

3.2 A concrete semantic-ABI failure

Consider two acceleration-structure primitives at physical positions 0 and 1 whose application identifiers are 10 and 20. Queries 100 and 101 should emit the pairs (100, 10) and

Table 2. Represented facts and their observable compiler consequences. D = source-derived; A = author-declared; F = fixed family rule; I = identity comparison. Shared machinery does not imply a generic device lowerer.

Program fact and origin	Representation	Consumer and consequence	Evidence boundary
Records and role signatures (D); three-u32 target (F)	Typed records, effects, flattened ABI	Verification checks signatures; generation emits leaf parameters, payload transfers, and output construction	CPU compilation/source trace; arbitrary layouts unsupported
Closest-hit metadata read (D); column association (A)	Indexed IR plus argument-to-column binding	Wrapper supplies the declared front/back column; rebinding changes generated call arguments	Existing CPU test; business meaning is not inferred
Slot-to-metadata source edit (D/I)	Changed IR identity, same effect digest	Old physical plan is rejected; rebuilding the plan is valid	Existing stale-plan test; no GPU result or proof of ID values
Complete relation (F); any-hit effects (D)	Family schema compared with all returned effects	Logical `IGNORE/TERMINATE` is rejected; overflow publishes no partial relation	Fixed route only; no general enumeration theory
Standard all-hit increment (D/F)	Exact-IR guard plus triangle route	Accepted hits update payload then physically call `optixIgnoreIntersection()`; +2 exits the intrinsic	Source-to-wrapper test and trusted code; no general optimization theorem

$(101, 20)$. If an intersection producer instead writes the physical position into attribute slot zero while the consumer interprets that slot as an application identifier, the apparent rows are $(100, 0)$ and $(101, 1)$. Every value remains a valid `u32`; width, slot, and row layout need not expose the semantic substitution. This is an illustrative witness, not a retained execution of the defective route.

The defect is not a machine-type error but a disagreement over what the bits mean. In RTDL, a trusted bounded-relation schema declares that attribute slot zero denotes an application identifier; the compiled relation contract carries that row source into a separately derived target projection. Admission compares the two compiler representations. Existing tests mutate only this fact and obtain `CP002_ATTRIBUTE_ABI_OWNERSHIP_MISMATCH`; with native initialization overlap disabled, an integrated projection mutation is rejected before the native-library loader is called. This proves check liveness, not automatic inference of application meaning or end-to-end execution of the illustrative defect. Both derivations remain in one compiler TCB, and a wrong but internally coherent trusted schema can pass.

3.3 Failure model

We distinguish four outcomes. *Reject* means admission fails before a launch. *Status failure* means execution reports failure and output is not published. *Wrong output* means a worker reports success but an independent oracle disagrees. *Unverified output* means a value exists without the required receipt or oracle evidence. The distinction prevents a status gate from being described as numerical correctness and prevents a generated plan from being described as an executable lowering.

A focused mock reproduced an unrepaired provider double fault: secondary bind or close failure can replace the primary exception and lose cleanup retry ownership; the mock returned no public result. A native-fork probe accepted a public call through a mock native implementation after bypassing the registered Python at-fork hook; it did not execute GPU work. Neither defect was observed in retained successful GPU workers, and inherited owners after such forks remain unsupported.

3.4 Typed callback representation

The source language is restricted Python, not arbitrary Python. Source is parsed through an AST allowlist and is not imported or executed for discovery. The front end resolves declared immutable records, frozen constants, same-unit acyclic helpers, and selected intrinsics, then produces canonical typed IR. Deserialization reconstructs typed objects and reruns verification; serialized IR is not itself an authority. Table 3 summarizes the representation.

The current resource contract admits at most 32 payload slots and eight attribute slots, trace depth one, callable depth zero, bounded helper depth, and statically bounded iteration. The numeric policy requires explicit binary32 behavior and fail-closed handling of prohibited integer overflow and nonfinite effects. These are language and resource checks; they do not prove that a chosen ray encoding computes the intended application function.

3.5 Role-indexed effects

Seven language roles participate in an admitted unit. `bounds` runs during preparation; `make_ray` and `finalize` are language leaves invoked by a trusted ray-generation wrapper; and `intersection`, `any_hit`, `closest_hit`, and `miss` map to target-stage behavior. User code cannot invoke raw traversal. Instead, each role returns one typed effect from a role-specific set: bounds returns an AABB, ray construction returns a trace request, intersection returns hit or no-hit, hit callbacks return payload plus continue/ignore/terminate control, miss returns payload, and finalize returns an output.

Topology makes these requirements relational. Custom AABBs require bounds, ray construction, intersection, miss, finalize, and at least one hit role. Built-in triangles forbid bounds and intersection because hardware owns them, but retain ray construction, miss, finalize, and at least one hit role. An any-hit role also requires an explicit delivery and continuation contract; the presence of that declaration does not prove order independence for arbitrary callback logic.

Table 3. Implemented bounded representation. Rejected forms have no fallback lowering.

Form	Accepted subset	Rejected boundary
Data	Fixed-width scalars; 2–4-lane vectors; tuples; acyclic immutable records; checked read-only views	Raw pointers, mutable/opaque objects, recursive records, dynamic allocation
Computation	Typed fields/indexing, arithmetic, selected intrinsics, bounded helpers/loops, two-sided conditionals, role-effect returns	Reflection, dynamic dispatch, foreign calls, recursion, exceptions, generators, async, dynamic loops
Unit and manifest	Records, constants, numeric/resource policy, topology, continuation, linkage, canonical source/IR/effect identities	Self-minted physical authority, unsupported budgets, malformed or stale identities, unverified deserialization

Role legality is necessary but not sufficient for a fixed result route. The general any-hit role admits continue, ignore, and terminate effects. The current complete bounded-relation verifier instead collects all return effects and requires only logical acceptance with continued traversal; it defines no filtering or early-termination meaning for that route. Thus a role-valid effect can be rejected because it conflicts with the route's output obligation. This is an implemented family restriction, not a claim that all complete-enumeration designs must reject filtering.

Logical effects also need not map one-to-one to physical intersection actions. In the triangle all-hit route, the logical continue effect first updates the payload and then physically ignores the intersection, preserving later events instead of shrinking traversal's accepted interval. Native geometry separately requests one any-hit delivery per primitive, and the reduction checks overflow. These are established OptiX mechanisms [24]; RTDL's contribution is to make their required combination part of the trusted implementation of a fixed result contract, not to invent all-hit traversal.

3.6 Fail-closed judgments

Expression checking derives $\Gamma \vdash e : \tau$. Statement checking also returns the output environment, all-paths-return bit, conservative static work, and observed effects:

$$\Gamma; r \vdash s \Rightarrow \langle \Gamma', q, n, \epsilon \rangle. \tag{1}$$

For each role r, verification enforces its exact signature and

$$M \vdash F_r : \epsilon \quad \text{with} \quad \epsilon \subseteq \mathsf{AllowedEffects}(r). \tag{2}$$

Names are single-assignment except for type-preserving updates inside static loops; loop locals cannot escape; both non-returning branches define the same typed names; and every role/helper path returns. Whole-unit checking validates schema/source identity, acyclic records and helpers, numeric and resource budgets, role cardinality, topology, linkage, and required external physical authority:

$$\mathcal{G}; \mathcal{R} \vdash Q \Downarrow \mathsf{Verified}(h_{\text{IR}}, h_{\text{effects}}). \tag{3}$$

Here $\mathcal{G}$ is an out-of-program geometry authority and $\mathcal{R}$ is the compiler-owned role topology. The judgment is explanatory notation for implemented checks, not a mechanized soundness theorem.

Admission is conjunctive. A route is accepted only if its schema is valid, its roles permit all effects, its semantic and physical ABIs agree, a trusted lowerer exists for its topology, the generated executable identity is bound, and the runtime lifecycle can reach the prepared state. Failure of any one predicate rejects the route; there is no "best effort" projection into a nearby executable.

3.7 Layered whole-protocol admission

Admission compares several authorities rather than trusting one manifest. Verified role topology and returned effects constrain the callback ABI; trusted family schemas give nominal meaning to otherwise ordinary machine fields; typed physical plans bind geometry, buffers, hit channels, reducers, and capacity; and materialization binds generated source/PTX and provider symbols to runtime state. Continuation and publication rules additionally require the selected status vocabulary and status-before-output order. The target projections are regenerated by the same compiler TCB, not independently proved. Hash equality establishes identity coherence rather than semantic correctness, and none of these comparisons synthesizes an arbitrary physical plan.

4 Code Generation and Runtime

4.1 Leaves, wrappers, and specialization

Verification produces one role-indexed ABI containing flattened inputs, outputs, effect tags, status tags, and identities. For the general route, the compiler emits deterministic Python source for each reachable role and helper, then invokes an isolated Numba compiler process to produce target-specific PTX device functions. The isolated child receives compiler-generated source rather than user modules and does not discover the active CUDA device. This leaf step does not build an OptiX pipeline.

A trusted topology wrapper composes the leaves with compiler-owned raygen, intersection, hit, and miss mechanics. It checks returned tags before issuing payload writes, `optixIgnoreIntersection()`, or termination. Thus the user effect `accept_continue` is a logical action; its physical action depends on the admitted route. Wrapper source, leaf PTX, ABI, provider symbols, and physical schema are hashed into the executable identity.

The evaluated standard routes also use trusted specializations guarded by exact callback IR and reducer identity. For a separate standard all-hit count program, the specialized

entry implements the known increment-and-continue behavior directly; the general path obtains the same declared behavior through the leaf ABI and wrapper. Bounded relation similarly fuses intersection and row emission. These specializations replace generic leaf calls and per-leaf tag interpretation at selected roles, but retain static schema, ABI, identity, overflow, capacity, status, and supplied expected-output checks. An exact-IR guard selects trusted code; it is not a proof that the specialization preserves source semantics.

Selection depends on program identity, not just role and ABI shape. An existing compiler test changes the standard increment from one to two. The program remains front-end legal and has the same role shape, but its IR identity changes and the generator exits the fixed count intrinsic for the general leaf path. This source-to-wrapper check establishes one selection boundary; it neither executes the modified program on a GPU nor proves arbitrary specialization correctness. The recognizer, wrapper, and specialized lowerer remain in the trusted computing base (TCB).

4.2 Plan versus executable lowering

The canonical planner normalizes the admitted schema and emits a declarative plan. That plan is intentionally marked non-executable. It does not synthesize arbitrary CUDA or OptiX callbacks. Instead, an accepted route names a compiler-owned, topology-specific lowerer and wrapper. The lowerer emits source and PTX, binds native symbols, constructs the program groups and shader-binding table, and records their identities. This design supplies a common admission surface while keeping concrete backend knowledge visible in the TCB.

Shared schema and lifecycle machinery do not make device code generic. The prospective sphere route in Section 6.3 required about 2,635 physical text lines in eight new topology-specific files, plus 23 additions and five deletions in one existing compiler file. These are diff counts, including comments and blanks, not effort or semantic LOC. The observation shows reuse of three sealed shared files in one extension, not automatic topology lowering.

4.3 Executable identity and lifecycle

The system binds hashes and shape metadata for generated source, PTX, ABI, native provider symbols, and the public route. The exact app-free native runtime may be loaded and warmed before route admission, including concurrently with AOT artifact verification. Admission still gates acceptance of a materialized or loaded route and its subsequent per-route preparation. The admitted route lifecycle is:

$$\text{static verify} \rightarrow \text{materialize/bind} \rightarrow \text{prepare} \rightarrow \text{execute/status gate} \rightarrow \text{output} \rightarrow \text{receipt}.$$

The materialized-program interface validates execution identity, native status, output digest, and traversal receipt before returning `ProtocolExecutionResult`. The measured AOT path instead returns `RTDLExecutionResult`: it synchronously rejects native and compact-status failures and any supplied expected-output mismatch, but an ordinary fast result may omit a digest and detailed receipt. The worker checks value and digest after prepared return; for first result this is after the prepared timer but inside the endpoint. It does not rehash disk files per launch. The preregistered per-execution receipt requirement is unmet: 4,096 timed A executions have only 32 separate post-loop diagnostic executions, one detailed receipt per A worker.

5 V4 Applications and Case Studies

RTDL was developed against repurposed-RT applications rather than rendering shaders alone. Table 4 reconstructs nine project-authored V4 mappings to algorithms credited to prior systems. RTDL contributes restricted expression, checking, generation, and composition of selected parts, not the application algorithm or geometric mapping. E1 means an exact historical application-source hash and callback-consumer binding survive. E2 means a frozen audit recorded the path and structural properties, but an independently retained historical per-file identity and the original 10.8-MB archive do not; recovered current entry points do not repair that custody gap. E2 is mapping evidence, not current runnability.

A frozen AST audit found no selected raw OptiX, CUDA, or PTX assembly tokens in the nine application files. This supports a structural shift in physical-owner preparation, not developer-time or LOC savings. Runtime loading passed through registered V4 interfaces for eight applications; RayDB is the exception, and trusted C++/CUDA/OptiX partners remain. The stable facade has two constructors, covers 2/6 OptiX build-input values and 2/4 leaf kinds, while the bounded corpus covers 4/6 values and four leaf kinds. Only one callback template is presently authorable, no independent user session was measured, and every new topology still requires a trusted lowerer.

6 Evaluation

We organize existing evidence around five questions. **RQ1** asks whether DSL and IR facts change admission, generation, or specialization. **RQ2** asks what historical mappings reuse and what remains application-owned. **RQ3** asks what a sealed extension reuses and what remains specific to a topology. **RQ4** asks what finite target properties an independent checker can validate. **RQ5** asks the runtime cost of exact and application paths. Evidence types remain separate: source traces and component tests are not GPU equivalence tests, historical mappings are not current runnable examples, finite mutations are not a soundness proof, and measured routes are not arbitrary-program lowering.

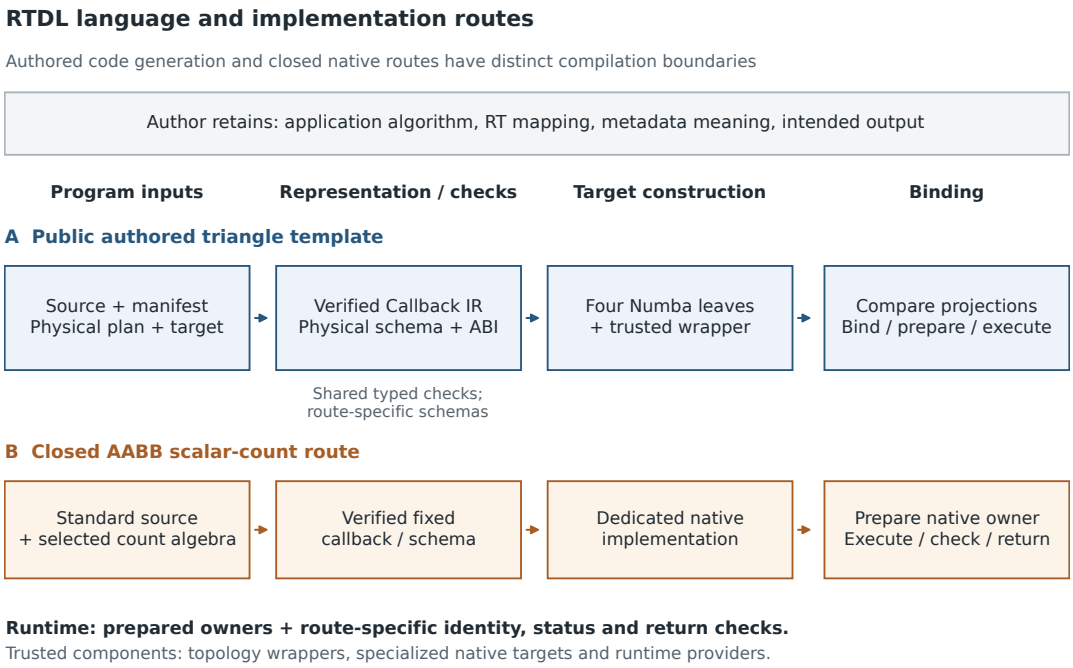

Figure 3. Two actual routes through RTDL. The public triangle template compiles authored roles into four Numba leaves and a trusted wrapper. The closed AABB scalar-count route validates a fixed callback/schema and selects a dedicated native implementation. Typed checks are partly shared; physical realization, identity checks, and prepared runtime behavior remain route-specific and trusted.

Table 4. Historical V4 mappings. Authors select the RT formulation and own the listed application semantics; E1/E2 are custody levels defined in text.

Application	Mapping and author-owned semantics	RTDL route and exact boundary
Particle [35]	Tetrahedral closest-face transition; cell encoding, transition, and I/O meaning	E1: built-in triangles, restricted closest-hit update, callback lowering, prepared lifecycle
Triangle [37]	RT-1A2/RT-2A1 selection; graph orientation and count meaning	E1: all-hit callback, checked capacity/status, segmented reduction; measured scalar is one algorithm stage
RayDB [30]	Partitioned grouped-I64 aggregate; relation, keys, partitions, and SQL meaning	E2: bounded emission and grouped reduction; sole private-loader exception
LibRTS [7]	AABB point/range containment; predicate, schema, and count/collect meaning	E1: typed custom-AABB relation and capacity/receipt checks; LibRTS is strong prior abstraction
X-HD [8]	Directed max-of-nearest witness; Hausdorff semantics and output	E2: cell-MBR traversal, checked state, tie-breaking, and reduction
RTNN [39]	Multiround distance-window top-k; metric, K, bounds, and ordering	E2: round/refit lifecycle and bounded selection; metric-specific traversal stays trusted
RT-DBSCAN [22]	Radius graph/components; radius, min-points, and clustering meaning	E2: bounded edges and grouped continuation; not every stage uses RT cores
RayJoin [6]	Six-batch planar overlay; schedule, predicates, and six full result tables	E2: typed producer/reducer route; recovered adapter returns summaries, not the registered full tables
RT-BarnesHut [21]	Aggregate hierarchy; tree, acceptance rule, and force law	E2: hierarchy/GAS ABI and frontier checks; tree traversal remains a trusted partner

6.1 Earlier liveness and residual evidence

For RQ1, the role-level any-hit set permits continue, ignore, and terminate, while the complete bounded-relation route accepts only continue returns. The triangle lowerer supplies the complementary transformation evidence: it records an accepted event before physically ignoring the intersection to preserve later events. These are source-traced implemented rules; this paper did not execute a deliberately terminating complete-relation program on the GPU.

Also for RQ1, we replayed an existing compiler test that changes the standard count update from +1 to +2. Front-end verification still succeeds, the callback IR identity changes, and wrapper generation switches from the fixed count intrinsic to the general leaf path. The test exercises parsing, IR verification, contract/ABI construction, and wrapper generation. It does not execute either generated path on a GPU or establish semantic preservation for other optimizations.

For two fixed contracts, 19 single-leaf mutations exercised 19/19 expected gates: seven role/effect, one semantic-ABI, seven physical-ABI, two continuation, and two identity mutations. This denominator covers only those contracts and mutations and is not pooled with later experiments.

With native-initialization overlap disabled, corrupting each target projection produced the expected sole reason before the native-library loader was called. For semantic ABI, this is mutation sensitivity, not an execution of the illustrative wrong-row route. Historical PyOptiX/OWL-style diagnostics lack independently verifiable execution records here and are not evidence about those systems' guarantees [25, 26]. The 19/19 result is neither soundness nor prevalence evidence.

6.2 Historical application evidence

For RQ2, the frozen source audit behind Table 4 classified all nine V4 mappings as a structural integration-responsibility shift and found registered-interface runtime loading in eight. It measured neither authoring time nor developer productivity. Three exact historical app/callback consumer identities remain independently recoverable. Current entry points have been recovered for all nine, but the other six retain only archive-level historical audit records. We therefore use the portfolio to show bounded mapping and composition diversity, not historical byte custody, current installability, or unseen-app success.

A historical RTX 4000 Ada authority reported 464 exact true-OptiX workers and 34 V2-direct/V4 rows: median criteria split 16/18 pass/fail, while 95% confidence intervals gave 11 V4 wins, 10 losses, and 13 uncertain. Its raw archive is excluded and was not rerun, so these results only prevent broad- performance inference; they are neither PyOptiX evidence nor pooled below.

6.3 Sealed-snapshot composition exam

For RQ3, before a public randomness pulse, we fixed an author-defined ten-row challenge: seven built-in four-role and three custom six-role rows. All used supported primitive families and returned one value per query: six produced counts, and four produced Booleans by terminating on the first accepted hit. The curve any-hit-terminate row remained eligible but unselected. The pulse selected `builtin_sphere::any_hit_count_continue_u64_per_query`.

Private custody identifies the sealed source. No bytes changed in three frozen schema, admission, and lifecycle files. The route made two OptiX launches and passed 12/12 rational-oracle rows, showing reuse of admission, identity, and lifecycle for this composition. It is not an unbiased application: authors defined the domain, rows recombined known ingredients, and the selected route required substantial topology-specific code.

6.4 Independent finite checker

For RQ4, the offline checker imports no RTDL module or compiler projection. It reads generated source, PTX/ABI metadata, symbols, and receipts. Across three route groups and four modes, it applied five property classes 20 times using 15 unique mutations, checking selected target-code patterns and ordering constraints.

This checker is finite, not a verifier for arbitrary device control flow. During review, an out-of-authority in-memory probe inserted an unconditional early return and still passed selected effect/status helper checks. The result therefore supports specialized target-structure validation only. General control-flow, numerical, and refinement soundness remain open; tools such as GPUVerify illustrate the substantially stronger proof obligation [2].

6.5 Performance methodology

For RQ5, we measured two exact tasks. *Relation* uses 4,096 objects and 4,096 queries, returns 4,096 canonical u32 pair rows, and performs two true OptiX traversals. *Triangle* uses 16,384 primitives and 16,384 queries, performs one true traversal, and returns the checked scalar 65,530.

Five arms isolate different boundaries:

- A current RTDL ahead-of-time public path at measured source M;
- B idiomatic pinned PyOptiX-compatible baseline;
- C strong PyOptiX-compatible baseline with device continuation;
- D named Direct CUDA/OptiX reference implementation;
- E frozen predecessor control for RTDL.

We ran the same registered matrix on an RTX 4090 (Ada) and RTX 3090 (Ampere). Each generation has eight blocks, two tasks, five arms, 80 cells, and 128 steady samples per cell: 160 cells and 20,480 samples total. Ratios are medians of eight within-block ratios of integer nanoseconds; lower is better, and raw times are not compared across machines. The steady timer encloses a prepared public action through return; validation follows. Entry starts before implementation-specific imports, while post-import starts after them; both end after first-result validation. PyOptiX initializes CUDA during import, whereas RTDL does so later, so these intervals include different lifecycle work.

The reported prepared latencies measure the disclosed exact-standard-IR triangle and relation specializations, not execution of every role through the general Numba-leaf ABI.

The registered primary criterion was prepared A/D median ≤ 1.20, with no maximum-block gate. A/C entry used median ≤ 1.20 and maximum ≤ 1.35, but the endpoint was revised after an adverse result and both first-result endpoints remain confounded. We report A/C only diagnostically. Only prepared A/E was a registered regression gate; A/E first-result values are post hoc. C/B competence used ≤ 1.05.

Table 5. Prepared public RTDL versus direct CUDA/OptiX (A/D). Ratio is lower-is-better; times are medians of worker medians.

GPU	Task	A (ms)	D (ms)	Median / max
RTX 4090	Relation	0.2810	0.2612	1.0769 / 1.0923
RTX 4090	Triangle	0.0598	0.0510	1.1751 / 1.2110
RTX 3090	Relation	0.2402	0.2198	1.0948 / 1.1188
RTX 3090	Triangle	0.0608	0.0536	1.1336 / 1.1427

6.6 Prepared execution results

Table 5 reports the main observation. The public AOT path is 1.077–1.175× the direct implementation by median. The triangle maximum block on Ada is 1.211×; because no maximum A/D gate was registered, it is reported as dispersion rather than converted into a pass/fail criterion.

Across 20 observations, AOT cache-hit medians were 58.3–78.2 ms, about 1.04–2.02% of cold time. Hits still include lookup, validation, and load and are outside the prepared endpoint.

6.7 Adverse lifecycle results

Every A/C post-import median is adverse, 1.560–1.837×, with a 2.377× worst block. Because entry was revised after the adverse result, neither endpoint supports a confirmatory claim. Against E, A also regresses by about 8–22% at entry and 16–31% post-import despite favorable prepared A/E, showing material pre-steady-state protocol and Python/native lifecycle work. For 4090 relation/triangle and 3090 relation/triangle respectively, the four post-import medians are 1.749, 1.560, 1.837, and 1.637; their observed maxima are 1.866, 1.639, 2.377, and 1.653. A/C entry ratios are 0.618–0.681 but remain lifecycle-confounded. A/E entry, post-import, and prepared ranges are 1.080–1.217, 1.163–1.305, and 0.584–0.922; only prepared A/E was registered, while the other A/E values are post hoc diagnostics.

Arm C outperformed B in all four formal steady comparisons (C/B ratios 0.221–0.654), establishing that B alone would be a weak baseline for this experiment. It does not establish C as globally optimal. We also recorded 1,024 timer-instrumentation endpoints for A only. Paired overhead was measured only for A; no corresponding paired overhead measurement was made for B, C, D, or E.

No wrong output was observed in the final GPU samples. This statement is not a correctness proof and does not repair the detailed-receipt shortfall described in Section 4.3.

6.8 Application-route cost against public PyOptiX

An initial frozen A_0/PyOptiX transaction on this RTX A4500 covered selected stages from three of nine mappings: Particle, RT-2A1, and LibRTS point/range. All twelve observations were adverse; Table 6 preserves every row rather than allowing the successor to erase them. Setup begins after shared domain preprocessing. "First" means the first checked execute after preparation, not process start or cold compilation. Prepared includes adapter checks and evidence projection, so these ratios are not intrinsic DSL tax.

Table 6. Initial A_0/public-PyOptiX paired-median ratios. Lower is better; every one of the 96 block ratios exceeds 1.

Unit	Setup + run	First execute	Prepared
Particle	10.208	75.533	37.928
com-dblp/1M	3.411	2.732	1.379
LibRTS point	1.312	12.059	38.467
LibRTS range	1.080	13.422	39.083

The old routes did different work. Particle used an any-hit candidate collector and logical leaf calls while its handwritten CUDA baseline used native closest-hit; Graph rebuilt per-segment V4 executor objects while the baseline retained its pipeline; LibRTS validated a fixed callback/schema and selected a dedicated native implementation rather than executing a newly generated Numba-leaf ABI. These are inspected route differences, not a measured causal decomposition of the slowdowns.

Remediation removed repeated compilation, materialization, and receipt work and reused identity-bound executable families. A new transaction was frozen before worker zero. Successor RTDL (A_s) and PyOptiX (C) both exclude offline compilation. Setup starts from the same derived inputs and includes deployment, one run, and close; prepared reuses state after one untimed warmup. Each row has eight paired fresh-process blocks, four per order; the registered envelope is median ≤ 1.20 and every block ≤ 1.35.

All 240 workers had distinct PIDs, equal outputs, CPU-8 affinity, and zero retry/discard/timeout; an independent recount reconstructed all 80 cells. The preselected cit-Patents row executes 28 segments and 69,850,749 physical queries: prepared A_s/C is 9.348/8.802 s and 1.057×; other rows are subsecond. Both campaigns exclude earlier domain loaders and offline encoding; the original raw-domain end-to-end objective therefore remains unfulfilled. Particle is a fixed app-shaped specialization; Graph and LibRTS use typed families but retain app-owned encoding and oracles. The result covers five units and one host condition, not the six unmeasured mappings or an app-independent engine.

7 Limitations and Broader Implications

Users provide bounded callback intent and data; shared RTDL machinery validates schemas, cross-role effects, ABI, identity, and lifecycle; trusted per-topology lowerers generate and bind CUDA/OptiX code. Failed status gates suppress output, while experiments may add application oracles.

This split does not make an arbitrary application correct. The author must select an RT formulation, encode geometry,

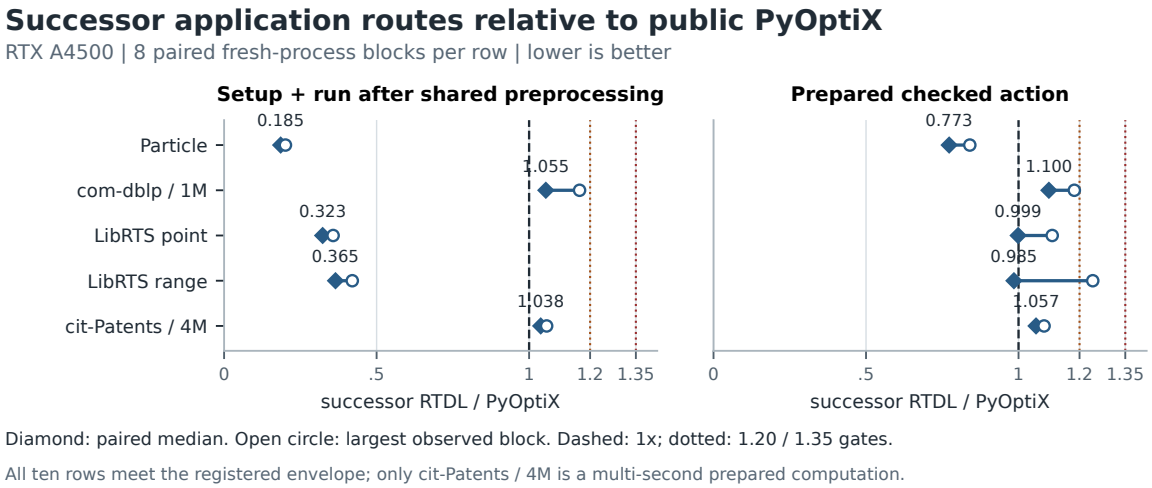

Figure 4. Successor RTX A4500 results. Diamonds are medians of eight paired block ratios; open circles are the largest observed blocks. All ten rows meet the separately registered 1.20-median/1.35-maximum engineering envelope. The figure reads the anonymous independently recounted projection; observed ranges are not confidence intervals.

declare trusted semantic meanings, provide domain predicates and reductions not covered by a fixed route, and validate the result against an independent oracle. The compiler can reject inconsistency among represented obligations, but a wrong yet internally coherent declaration can pass. Nor does verified Callback IR imply an executable path: only topologies with audited lowerers can be materialized.

The language trades expressiveness for analysis. It excludes dynamic allocation, recursion, reflection, unrestricted loops, raw pointers, and arbitrary foreign calls. Its stable facade has two constructors. The two-generation synthetic paths use exact-IR specializations; the application study also measures disclosed callback-leaf and closed native routes. No independent developer has measured authoring effort. These are design costs, not temporary omissions that the evaluation silently treats as solved.

The implementation suggests, but does not validate elsewhere, a pattern for managed-language/accelerator boundaries: use the application-visible result contract to organize callback legality, target-action interpretation, and publication; then distinguish metadata checks from trusted lowering obligations and execution-time failures. Other accelerators may expose analogous splits, but their effects, lowerers, and checker vocabularies would need independent design and evidence.

8 Related Work

8.1 Repurposed RT applications and abstractions

A recent review covers 35 RT solutions across 32 problems [19]. RayJoin, RayDB, RTNN, RT-DBSCAN, triangle counting, X-HD, particle tracking, and RT-BarnesHut each supply an application-specific mapping and result protocol [6, 8, 21, 22, 30, 35, 37, 39]. RTDL does not claim their encodings, continuation, deduplication, nearest-K maintenance, or orientation rules. LibRTS is the closest application-facing abstraction: it offers point/range queries and user device handlers for count and collection [7]. RTDL supports fewer operations but adds restricted role source, result-route checks, and executable-identity binding; we claim neither user-level superiority nor that LibRTS cannot add them. TTA/TTA+ changes hardware for general tree traversal [10]; RTDL instead accepts fixed-function OptiX and an author-selected mapping.

8.2 RT languages and compilation systems

OptiX established programmable RT [27]; current OptiX, Vulkan, and DXR define substantial pipeline, payload, interface, and SBT rules [16, 17, 20, 24]. PyOptiX supports Python host code, Numba-authored device programs, and OptiX intrinsics, while OWL owns pipeline construction [25, 26, 36]. Its pinned example explicitly builds parameter layout, program groups, pipeline, SBT, and launch; RTDL derives a role ABI and wrapper arguments from checked source and plan. This allocates responsibility differently, not impossibility to PyOptiX.

Shader Components/Slang provide modular interfaces, specialization, reflection, capabilities, and typed binding; SlangPy adds Python RT pipeline and lifecycle support [11, 12, 15, 31–33]. Luisa, CrossRT, and Dr.Jit automate or compile broader host/device and RT work [4, 14, 18, 38]. Rendering DSLs also encode domain contracts [1]. These systems preclude first-to-automate claims.

Bonsai derives pruning and subtree-inclusion conditions from query predicates and annotated tree metadata and fuses query operators into generated tree traversals [29]. It therefore treats query-dependent traversal more automatically than RTDL's fixed family rules. Scion's object is a logical tree, physical layout, and traversal algorithm [9]. RTDL instead takes an author-selected RT mapping as given and carries a bounded result protocol across OptiX's split roles, physical data interfaces, generated target identity, and fail-closed publication. This is the concrete compiler object we implement, not an inability claim about either neighboring system.

For fixed families, that object links result obligations to role effects, semantic and physical contracts, trusted traversal code, executable identity, and interface failures. Restricted expressiveness and topology-specific trusted code are its cost; the individual techniques are not claimed as new.

8.3 Protocol and validation foundations

Typestate, interface automata, session types, typed linking, and FFI checking provide more general or formally stronger protocol and representation models [3, 5, 13, 28, 34]. Proof-carrying code validates supplied proofs before native execution [23]. RTDL offers neither a new linking calculus nor a proof certificate: schemas, projections, lowerers, and runtime remain trusted, and hashes prove identity rather than semantics. Its finite structural checker is not formal GPU verification [2].

9 Artifact

F2's nine-member anonymous projection and standard-library verifier recount 160 cells, 20,480 steady samples, 1,024 A-only instrumentation endpoints, 20 AOT samples, and eight competence records; twin clean exports were byte-identical and replayed under normal and optimized Python. Separately, a private 1,049-file successor archive supports clean-checkout reconstruction, and its seven-file anonymous projection recounts all 240 workers, 1,248 samples, 120 warmups, 80 paired cells, and ten evaluations while removing private paths, host, GPU UUID, PIDs, and source identities. Both packages recount offline: they neither rerun GPU work, include every raw input, nor replace private custody.

10 Threats to Validity

Construct validity. Prepared A/D excludes authoring, cold compilation, and first use. Direct D is a named contract-equivalent implementation, not a proven lower bound; strong C is output-equivalent but not operation-identical; paired instrumentation covers A only. Application setup follows shared loaders/preprocessing, so the raw-domain end-to-end objective is unfulfilled. Offline compilation is excluded from both successor arms, but routes and timed validation differ. Semantic-ABI mutations show check liveness, not a defective GPU execution; a coherent trusted schema may still encode the wrong meaning.

Internal validity. Both synthetic tasks were adapted, not unseen. Entry changed after an adverse result; both first-result endpoints remain confounded. Mocks expose provider double-fault and native-fork defects but ran no GPU work. CPU 8 followed a disclosed diagnosis; an unpooled 48-CPU scan reversed its Particle ratio from formal .773 to 1.400, so sub-ms rows are not stable guarantees. GPU clocks were unlocked.

External validity. Synthetic measurements cover two NVIDIA generations and two exact tasks; raw times are not compared across machines. Application evidence covers selected stages from 3/9 apps, four original units, one long graph scale, and one RTX A4500; six apps are unexecuted, inputs derived, and peak-memory, independent-user, prevalence, and arbitrary-Python/IR evidence absent. Six recovered entry points lack historical per-file custody; older timings are not pooled with M.

Implementation trust. The sphere route required about 2,635 physical lines in eight new topology-specific files plus a 23-addition/5-deletion compiler diff; the finite checker missed an early-return probe; 4,096 timed A calls lack per-call receipts. The application remediation adds trusted executable-family and admission code; Particle still relies on an app-shaped fixed standard-library native route.

Artifact scope. F2 and the separate application projection check disclosed identities, counts, formulas, and rows but cannot recreate GPU runs or private custody. The latter cross-binds absent private bytes by hash rather than validating their contents.

11 Conclusion

RTDL connects bounded restricted-Python roles and result contracts to trusted physical actions, executable identity, and fail-closed publication for author-selected mappings; mappings and oracles stay external. Templates emit Numba leaves plus trusted wrappers; closed routes select native code. Rules reject incompatible effects, preserve all-hit continuation, and exit exact specializations after semantic changes. Nine mappings show reuse and application responsibility, not generality or usability. Across two GPUs, prepared standard routes cost 1.077–1.175× Direct; adverse post-import behavior reaches 1.837× median/2.377× maximum, and all twelve initial app observations are adverse. Ten successor rows meet their registered 1.20/1.35 envelope, bounding tested-route costs only.

References

[1] Academy Software Foundation. 2026. Open Shading Language Documentation. https://open-shading-language.readthedocs.io/en/latest/. Programmable shading for advanced renderers.

[2] Adam Betts, Nathan Chong, Alastair F. Donaldson, Shaz Qadeer, and Paul Thomson. 2012. GPUVerify: A Verifier for GPU Kernels. In *Proceedings of the ACM International Conference on Object-Oriented Programming Systems Languages and Applications*. ACM, New York, NY, USA, 113–132. https://doi.org/10.1145/2384616.2384625

[3] Luca de Alfaro and Thomas A. Henzinger. 2001. Interface Automata. In *Proceedings of the 8th European Software Engineering Conference Held Jointly with the 9th ACM SIGSOFT International Symposium on Foundations of Software Engineering*. ACM, New York, NY, USA, 109–120. https://doi.org/10.1145/503209.503226

[4] Vladimir Frolov, Vadim Sanzharov, Albert Garifullin, Maxim Raenchuk, and Alexei Voloboy. 2024. CrossRT: A Cross Platform Programming Technology for Hardware-Accelerated Ray Tracing in CG and CV Applications. arXiv:2409.12617. https://doi.org/10.48550/arXiv.2409.12617

[5] Michael Furr and Jeffrey S. Foster. 2005. Checking Type Safety of Foreign Function Calls. In *Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation*. ACM, New York, NY, USA, 62–72. https://doi.org/10.1145/1065010.1065019

[6] Liang Geng, Rubao Lee, and Xiaodong Zhang. 2024. RayJoin: Fast and Precise Spatial Join. In *Proceedings of the 38th ACM International Conference on Supercomputing*. 124–136. https://doi.org/10.1145/3650200.3656610

[7] Liang Geng, Rubao Lee, and Xiaodong Zhang. 2025. LibRTS: A Spatial Indexing Library by Ray Tracing. In *Proceedings of the 30th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*. 396–411. https://doi.org/10.1145/3710848.3710850

[8] Liang Geng, Zhehu Yuan, Rubao Lee, Fusheng Wang, and Xiaodong Zhang. 2026. X-HD: Fast Hausdorff Distance Computation with Ray Tracing. In *Proceedings of the 40th ACM International Conference on Supercomputing*. 945–959. https://doi.org/10.1145/3797905.3800509

[9] Christophe Gyurgyik, Alexander J. Root, and Fredrik Kjolstad. 2026. Decoupling Data Layouts from Bounding Volume Hierarchies. *Proceedings of the ACM on Programming Languages* 10, PLDI, Article 175 (2026), 39 pages. https://doi.org/10.1145/3808253

[10] Dongho Ha, Lufei Liu, Yuan Hsi Chou, Seokjin Go, Won Woo Ro, Hung-Wei Tseng, and Tor M. Aamodt. 2024. Generalizing Ray Tracing Accelerators for Tree Traversals on GPUs. In *Proceedings of the 57th IEEE/ACM International Symposium on Microarchitecture*. 1041–1057. https://doi.org/10.1109/MICRO61859.2024.00080

[11] Yong He, Kayvon Fatahalian, and Theresa Foley. 2018. Slang: Language Mechanisms for Extensible Real-Time Shading Systems. *ACM Transactions on Graphics* 37, 4, Article 141 (2018), 13 pages. https://doi.org/10.1145/3197517.3201380

[12] Yong He, Theresa Foley, Teguh Hofstee, Haomin Long, and Kayvon Fatahalian. 2017. Shader Components: Modular and High Performance Shader Development. *ACM Transactions on Graphics* 36, 4, Article 100 (2017), 11 pages. https://doi.org/10.1145/3072959.3073648

[13] Kohei Honda, Nobuko Yoshida, and Marco Carbone. 2016. Multiparty Asynchronous Session Types. *J. ACM* 63, 1, Article 9 (2016), 67 pages. https://doi.org/10.1145/2827695

[14] Wenzel Jakob, Sébastien Speierer, Nicolas Roussel, and Delio Vicini. 2022. Dr.Jit: A Just-In-Time Compiler for Differentiable Rendering. *ACM Transactions on Graphics* 41, 4 (2022), 1–19. https://doi.org/10.1145/3528223.3530099

[15] Simon Kallweit, Chris Cummings, Benedikt Bitterli, Sai Bangaru, and Yong He. 2026. SlangPy v0.43.1. https://github.com/shader-slang/slangpy/releases/tag/v0.43.1. Release source commit 2f6c4625fdd2b3bd812ca6cd2802cf98bd89b248.

[16] Khronos Group. 2026. Vulkan Specification: Ray Tracing and Shader Binding Tables. https://docs.vulkan.org/spec/latest/chapters/raytracing.html.

[17] Khronos Group. 2026. Vulkan Specification: Shader Interfaces. https://docs.vulkan.org/spec/latest/chapters/interfaces.html.

[18] LuisaGroup. 2026. LuisaCompute: High-Performance Rendering Framework on Stream Architectures. https://github.com/LuisaGroup/LuisaCompute.

[19] Enzo Meneses, Cristóbal A. Navarro, Héctor Ferrada, Konstantin Verichev, and Cristian Salazar-Concha. 2027. Ray Tracing Cores for General-Purpose Computing: A Literature Review. *Future Generation Computer Systems* 186, Article 108739 (2027). https://doi.org/10.1016/j.future.2026.108739 Available online 3 August 2026.

[20] Microsoft. 2026. DirectX Raytracing Functional Specification: Payload Access Qualifiers. https://microsoft.github.io/DirectX-Specs/d3d/Raytracing.html.

[21] Vani Nagarajan, Rohan Gangaraju, Kirshanthan Sundararajah, Artem Pelenitsyn, and Milind Kulkarni. 2025. RT-BarnesHut: Accelerating Barnes–Hut Using Ray-Tracing Hardware. In *Proceedings of the 30th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*. 43–56. https://doi.org/10.1145/3710848.3710885

[22] Vani Nagarajan and Milind Kulkarni. 2023. RT-DBSCAN: Accelerating DBSCAN Using Ray Tracing Hardware. In *2023 IEEE International Parallel and Distributed Processing Symposium*. 963–973. https://doi.org/10.1109/IPDPS54959.2023.00100

[23] George C. Necula and Peter Lee. 1996. Safe Kernel Extensions Without Run-Time Checking. In *Proceedings of the 2nd USENIX Symposium on Operating Systems Design and Implementation*. USENIX Association, Seattle, WA, 229–243. https://www.usenix.org/conference/osdi-96/safe-kernel-extensions-without-run-time-checking

[24] NVIDIA. 2025. *NVIDIA OptiX 9.0 Programming Guide*. NVIDIA. https://github.com/NVIDIA/optix-sdk/blob/v9.0.0/doc/OptiX_Programming_Guide_9.0.0.pdf Payload types and per-stage payload semantics.

[25] NVIDIA. 2026. PyOptiX: Complete Python Bindings for the OptiX Host API. https://github.com/NVIDIA/otk-pyoptix. Pinned experiment source commit 3144f224c0fd18733925faf3d8fb82c7376b8dcf.

[26] NVIDIA, Ingo Wald, Stefan Zellmann, and contributors. 2026. OWL: The OptiX Wrappers Library. https://github.com/NVIDIA/OWL.

[27] Steven G. Parker, James Bigler, Andreas Dietrich, Heiko Friedrich, Jared Hoberock, David Luebke, David McAllister, Morgan McGuire, Keith Morley, Austin Robison, and Martin Stich. 2010. OptiX: A General Purpose Ray Tracing Engine. *ACM Transactions on Graphics* 29, 4, Article 66 (2010), 13 pages. https://doi.org/10.1145/1778765.1778803

[28] Daniel Patterson and Amal Ahmed. 2017. Linking Types for Multi-Language Software: Have Your Cake and Eat It Too. In *2nd Summit on Advances in Programming Languages (Leibniz International Proceedings in Informatics, Vol. 71)*. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, Dagstuhl, Germany, 12:1–12:15. https://doi.org/10.4230/LIPIcs.SNAPL.2017.12

[29] Alexander J. Root, Christophe Gyurgyik, Purvi Goel, Kayvon Fatahalian, Jonathan Ragan-Kelley, Andrew Adams, and Fredrik Kjolstad. 2026. Bonsai: Compiling Queries to Pruned Tree Traversals. *Proceedings of the ACM on Programming Languages* 10, PLDI, Article 178 (2026), 29 pages. https://doi.org/10.1145/3808256

[30] Xuri Shi, Kai Zhang, X. Sean Wang, Xiaodong Zhang, and Rubao Lee. 2025. RayDB: Building Databases with Ray Tracing Cores. *Proceedings of the VLDB Endowment* 19, 1 (2025), 43–55. https://doi.org/10.14778/3772181.3772185

[31] Slang Project. 2026. Capabilities—Slang User's Guide. https://docs.shader-slang.org/en/stable/external/slang/docs/user-guide/05-capabilities.html.

[32] Slang Project. 2026. A Practical and Scalable Approach to Cross-Platform Shader Parameter Passing Using Slang's Reflection API.

https://docs.shader-slang.org/en/stable/shader-cursors.html.

[33] SlangPy Project. 2026. SlangPy API Reference and Changelog. https://slangpy.shader-slang.org/en/stable/src/api_reference.html.

[34] Robert E. Strom and Shaula Yemini. 1986. Typestate: A Programming Language Concept for Enhancing Software Reliability. *IEEE Transactions on Software Engineering* SE-12, 1 (1986), 157–171. https://doi.org/10.1109/TSE.1986.6312929

[35] Bin Wang, Ingo Wald, Nate Morrical, Will Usher, Lin Mu, Karsten Thompson, and Richard Hughes. 2022. An GPU-Accelerated Particle Tracking Method for Eulerian–Lagrangian Simulations Using Hardware Ray Tracing Cores. *Computer Physics Communications* 271 (2022), 108221. https://doi.org/10.1016/j.cpc.2021.108221

[36] Michael Yh Wang. 2022. Writing Ray Tracing Applications in Python Using the Numba Extension for PyOptiX. https://developer.nvidia.com/blog/writing-ray-tracing-apps-in-python-using-numba-for-pyoptix/. NVIDIA Technical Blog. Accessed September 7, 2026.

[37] Zhixiong Xiao, Mengbai Xiao, Yuan Yuan, Dongxiao Yu, Rubao Lee, and Xiaodong Zhang. 2025. A Case Study for Ray Tracing Cores: Performance Insights with Breadth-First Search and Triangle Counting in Graphs. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 9, 2, Article 16 (2025), 25 pages. https://doi.org/10.1145/3727108

[38] Shaokun Zheng, Zhiqian Zhou, Xin Chen, Difei Yan, Chuyan Zhang, Yuefeng Geng, Yan Gu, and Kun Xu. 2022. LuisaRender: A High-Performance Rendering Framework with Layered and Unified Interfaces on Stream Architectures. *ACM Transactions on Graphics* 41, 6, Article 232 (2022), 19 pages. https://doi.org/10.1145/3550454.3555463

[39] Yuhao Zhu. 2022. RTNN: Accelerating Neighbor Search Using Hardware Ray Tracing. In *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*. 76–89. https://doi.org/10.1145/3503221.3508409